

NFA in Fig. 2.35 is NFA for the language L .

Example 2.15. Find the NFA with four state for the language

$$L = \{a^n : n \geq 0\} \cup \{b^n a : n \geq 1\}.$$

Solution. Again we will adopt the same approach as in example 2.14.

$$L = L_1 \cup L_2$$

$$L_1 = a^n : n \geq 0$$

$$L_2 = b^n a : n \geq 1$$

NFA for the language L_1 is

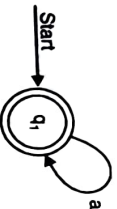


Fig. 2.36.

NFA for the language L_2 is

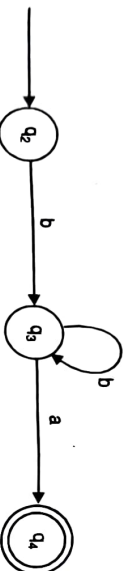


Fig. 2.37.

Now combine NFA in Fig. 2.36 and Fig. 2.37 as

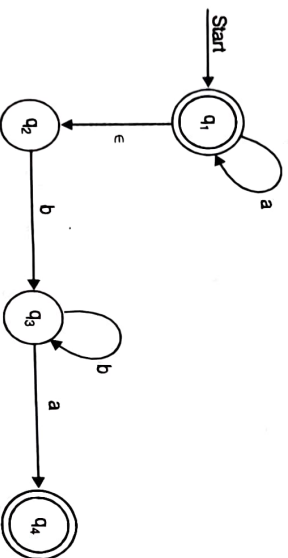


Fig. 2.38. NFA for L .

Finally Fig. 2.38 is the NFA for the language L .

Extended Transition Function for NFA

Similar to DFA's, we need to ... and the transition function δ of an NFA to a function $\hat{\delta}$ that takes a state and string and return the set of states.

Basis. $\hat{\delta}(q, \epsilon) = \{q\}$. That is without reading any input symbols, we are only in the same state.

Induction. Suppose w is of the form $w = xa$, where a is the last symbol and x is the substring containing rest of w . Let us suppose that

$$\hat{\delta}(q, x) = \{p_1, p_2, \dots, p_k\},$$

$$\text{Let } \bigcup_{i=1}^k \hat{\delta}(p_i, a) = \{r_1, r_2, r_3, \dots, r_m\}$$

Then $\hat{\delta}(q, w) = \{r_1, r_2, \dots, r_m\}$. Less formally, we compute $\hat{\delta}(q, w)$ by first computing $\hat{\delta}(q, x)$, and by then following any transition from any of these states that is labeled a . Let us see an example for better understanding of the concept.

Example 2.16. Consider NFA in Fig. 2.39, and describe the processing of string 00101 by it.

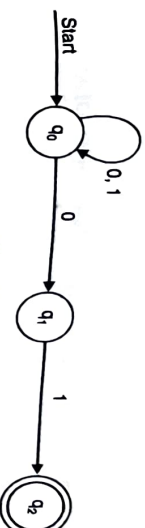


Fig. 2.39.

Solution. It is NFA accepting all strings end in 01. Let us use $\hat{\delta}$ to describe the processing of input 00101 by NFA. A summary of the steps is.

Step 1. $\hat{\delta}(q_0, \epsilon) = \{q_0\}$

Step 2. $\hat{\delta}(q_0, 0) = \hat{\delta}(q_0, 0) = \{q_0, q_1\}$

Step 3. $\hat{\delta}(q_0, 00) = \hat{\delta}(q_0, 0) \cup \hat{\delta}(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$

Step 4. $\hat{\delta}(q_0, 001) = \hat{\delta}(q_0, 1) \cup \hat{\delta}(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$

Step 5. $\hat{\delta}(q_0, 0010) = \hat{\delta}(q_0, 0) \cup \hat{\delta}(q_2, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$

Step 6. $\hat{\delta}(q_0, 00101) = \hat{\delta}(q_0, 1) \cup \hat{\delta}(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$.

Step 1 is the basis rule. We obtain step 2 by applying $\hat{\delta}$ to the done state q_0 that is in the previous set, and get $\{q_0, q_1\}$ as a result. Step 3 is obtained by taking the union over two states in previous set of what we get when we apply $\hat{\delta}$ to the input 0. That is, $\hat{\delta}(q_0, 0) = \{q_0, q_1\}$, while $\hat{\delta}(q_1, 0) = \emptyset$.

For step 4, we take union of $\hat{\delta}(q_0, 1) = \{q_0\}$ and $\hat{\delta}(q_1, 1) = \{q_2\}$. So step 5 and step 6 are obtained in similar way.

2.10. THE LANGUAGE OF NFA

As we discussed, an NFA accepts a string w if it is possible to make any sequence of choice of next state while reading the characters of w , and go from the start state to any accepting state.

If M is a NFA

$$M = (Q, \Sigma, \delta, q_0, F) \text{ then}$$

$L(M) = \{w / \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$ is the Language of M i.e. NFA. $L(M)$ is the set of strings w in Σ^* such that $\hat{\delta}(q_0, w)$ contains at least one accepting state.

2.11. TRANSFORMATION OF NFA TO DFA

For every non deterministic finite automata there exist an equivalent deterministic finite automata. The equivalence is defined in terms of language acceptance. Since a NFA is a nothing but a finite automata in which zero, one or more transition on an input symbol is permitted, we can always construct a finite automata which will simulate all the moves of NFA on a particular input symbol in parallel, then get a finite automata in which there will be exactly one transition on every input symbol, hence it will be DFA equivalent to NFA.

Since the DFA equivalent of NFA is to simulate the moves in parallel state of a DFA will be combination of one or more states of NFA, hence every state of a DFA will be represented by some subset of set of states of NFA, and therefore the transition of NFA to DFA is normally called a subset construction.

So transformation of NFA to DFA in value, finding subset of set of states of NFA. Let us implement this concept in the form of set of steps as algorithm through an example.

Example 2.17. Convert the following NFA in to DFA.

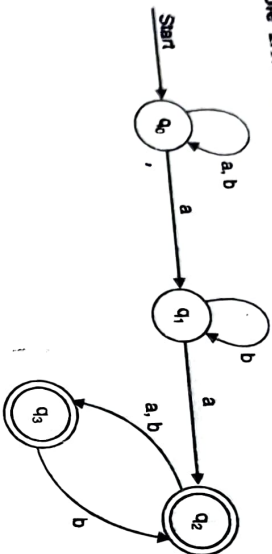


Fig. 2.40.

Solution.

Step 1. Seek all the transition from starting state q_0 for every symbol in Σ i.e. $\{a, b\}$. If you get a set of states for same input then consider that set as new single state as :

$$\delta(q_0, a) = \{q_0, q_1\}$$

in Fig. 2.40 q_0 transit to both q_0 and q_1 for input a .

$$\delta(q_0, b) = \{q_0\}$$

Step 2. In step 1 we are getting a new state $\{q_0, q_1\}$. Now repeat step 1 for this new state only i.e. check all transitions of a and b (that is Σ) from $\{q_0, q_1\}$ as :

$$\begin{aligned} \bullet \delta(\{q_0, q_1\}, a) &= \delta(q_0, a) \cup \delta(q_1, a) = \{q_0, q_1\} \cup \{q_2\} \\ &= \{q_0, q_1, q_2\} \end{aligned}$$

$$\delta(\{q_0, q_1\}, b) = \delta(q_0, b) \cup \delta(q_1, b) = \{q_0, q_1\}$$

Step 3. Repeat step 2 till you are getting any new state. All those states which consists at least one accepting state of given NFA as member state will be considered as final states.

$\{q_0, q_1, q_2\}$ is new state.

$$\begin{aligned} \delta(\{q_0, q_1, q_2\}, a) &= \delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a) \\ &= \{q_0, q_1\} \cup \{q_2\} \cup \{q_3\} \\ &= \{q_0, q_1, q_2, q_3\} \end{aligned}$$

(New state)

$$\delta(\{q_0, q_1, q_2\}, b) = \delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b)$$

$$= \{q_0\} \cup \{q_1, q_2\} \cup \{q_3\}$$

(New state)

$$\delta(\{q_0, q_1, q_2, q_3\}, a) = \{q_0, q_1, q_2, q_3\}$$

(Old state)

$$\delta(\{q_0, q_1, q_2, q_3\}, b) = \{q_0, q_1, q_2, q_3\}$$

(Old state)

$$\delta(\{q_0, q_1, q_3\}, a) = \{q_0, q_1, q_2, q_3\}$$

(Old state)

$$\delta(\{q_0, q_1, q_3\}, b) = \{q_0, q_1, q_2, q_3\}$$

(Old state)

Let draw transition table

δ/Σ	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$
$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_0, q_1, q_3\}$	$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$

Let us say

$$q_0 \rightarrow A$$

$$\{q_0, q_1\} \rightarrow B$$

$$\{q_0, q_1, q_2\} \rightarrow C$$

$$\{q_0, q_1, q_2, q_3\} \rightarrow D$$

$$\{q_0, q_1, q_3\} \rightarrow E$$

A is initial state and C, D and E are final states since they contain q_2 and q_3 as member which are final states of NFA.

So

δ/Σ	a	b
$\rightarrow A$	B	A
B	C	B
$*C$	D	E
$*D$	D	D
$*E$	D	D

Let us draw transition diagram

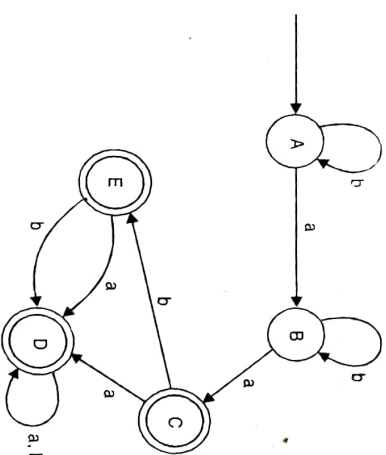


Fig. 2.41. DFA equivalent of given NFA.

It is a DFA.

Example 2.17. Given NFA is

δ/Σ	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_2\}$
q_1	$\{q_0\}$	$\{q_1\}$
$*q_2$	ϕ	$\{q_0, q_1\}$

Convert it in to DFA.

Solution. Do the same process as in example 2.16.

$$\begin{aligned}
 \delta(q_0, a) &= \{q_0, q_1\} & (\text{New}) \\
 \delta(q_0, b) &= \{q_2\} & (\text{New}) \\
 \delta(q_0, a) &= \delta(q_0, a) \cup \delta(q_1, a) = \{q_0\} \cup \{q_0\} & (\text{Old}) \\
 \delta(q_0, b) &= \delta(q_0, b) \cup \delta(q_1, b) = \{q_2\} \cup \{q_2\} & (\text{Old}) \\
 \delta(q_1, a) &= \{q_1\} & (\text{New}) \\
 \delta(q_1, b) &= \{q_1, q_2\} & (\text{New}) \\
 \delta(q_1, a) &= \{q_1\} & (\text{Empty set}) \\
 \delta(q_2, a) &= \{q_2\} & (\text{Old}) \\
 \delta(q_2, b) &= \{q_0, q_1\} & (\text{Old}) \\
 \delta(q_2, a) &= \delta(q_2, a) \cup \delta(q_1, a) = \{q_2\} \cup \{q_2\} & (\text{Old}) \\
 \delta(q_2, b) &= \delta(q_2, b) \cup \delta(q_1, b) = \{q_2\} \cup \{q_0, q_1\} & (\text{Old}) \\
 \delta(q_1, a) &= \{q_1\} & (\text{Initial state}) \\
 q_0 &\rightarrow A & (\text{Initial state}) \\
 q_2 &\rightarrow B & (\text{Final state}) \\
 q_0 &\rightarrow C & (\text{Final state}) \\
 q_1, q_2 &\rightarrow D & (\text{Final state})
 \end{aligned}$$

Transform table for DFA

δ/Σ	a	b
$\rightarrow A$	C	B
$*B$	$\{\phi\}$	C
C	C	D
$*D$	A	C

Transition diagram for DFA

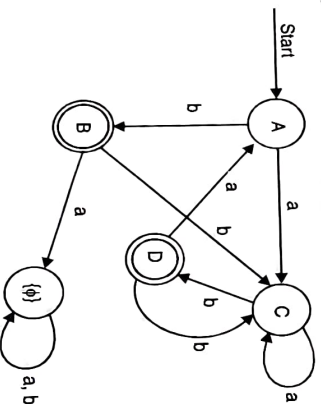


Fig. 2.42.

Here $\{\phi\}$ state is nothing but interpreted as if there is a input 'a' on state B then there is no path for it and machine will be hang.

2.12. NFA WITH ϵ -TRANSITIONS

If a FA is modified to permit transition without input symbols, along with zero, one or more transition on input symbols, then we get a NFA with ϵ -transitions, because the transition made without symbols are called as ϵ -transitions.

Consider the NFA given by following diagram.

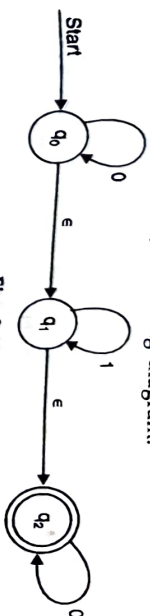


Fig. 2.43.

This is a NFA with ϵ -transitions because it is possible to make transition from state q_0 to q_1 without consuming any of the input symbols. Being a finite Automata, NFA with ϵ -transitions will also be denoted as five tuple.

$$M = (Q, \Sigma, \delta, q_0, F)$$

where Q, Σ, q_0 and F are having usual meaning, and transition function δ defines a mapping from

$$Q \times (\Sigma \cup \epsilon) \text{ to } 2^Q.$$

2.13. ACCEPTANCE OF A STRING BY NFA WITH ϵ -TRANSITIONS

A string w in Σ^* will be accepted by NFA with ϵ -transition, if there exist at least one path corresponding w , which starts in an initial state, and ends in one of the final states. Since this path may formed by ϵ -transition as well non ϵ -transitions to find out whether w is accepted or not by the NFA with ϵ moves, we require to defined a function ϵ -closure (q), where q is a state of the automata.

The function ϵ -closure (q) is defined as follows :

ϵ -closure (q) = set of all those states of the automata (NFA with ϵ -transitions) which can be reached from q on a path labeled by ϵ i.e. without consuming any input symbol.

Consider the NFA discussed in Fig. 2.43 then

$$\begin{aligned}
 \epsilon\text{-closure}(q_0) &= \{q_0, q_1, q_2\} \\
 \epsilon\text{-closure}(q_1) &= \{q_1, q_2\} \\
 \epsilon\text{-closure}(q_2) &= \{q_2\}
 \end{aligned}$$

2.14. CONVERSION OF NFA WITH ϵ -TRANSITIONS TO NFA WITHOUT ϵ -TRANSITIONS

There is a conversion algorithm from NFA with ϵ -transition s to a NFA without ϵ -transitions !

We will learn this algorithm by solving an example.

Example 2.18. Consider the NFA with ϵ -transition $M = (Q, \Sigma, \delta, q_0, F)$

$$\begin{aligned}
 Q &= \{q_0, q_1, q_2\} \\
 \Sigma &= \{a, b, c\} \text{ and } \epsilon \text{ moves.} \\
 \text{Initial state} &= \{q_0\} \\
 F &= \{q_2\}
 \end{aligned}$$

Transition Table

δ/Σ	a	b	c	ϵ
$\rightarrow q_0$	$\{q_0\}$	$\{\phi\}$	$\{\phi\}$	$\{q_1\}$
q_1	$\{\phi\}$	$\{q_2\}$	$\{\phi\}$	$\{q_2\}$
$*q_2$	$\{\phi\}$	$\{\phi\}$	$\{q_2\}$	$\{\phi\}$

and transition diagram is following

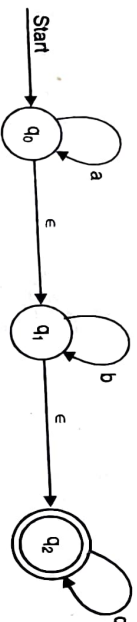


Fig. 2.44. NFA with ϵ -transition.

Solution.

Step 1. Find the states of NFA without ϵ -transition including initial state and final states as follows :

- (i) The language accepted by the above NFA with ϵ -transform is the set of strings over $\{a, b, c\}$ including the null string and all strings with any number of a 's followed by any number of b 's and follows by any number of c 's.
- (ii) There will be the same number of states but the names can be constructed by writing the state name as the set of states in the ϵ -closure.
- (iii) Initial state will be ϵ -closure of initial state of NFA with ϵ -transition in Fig. 2.44 as
 ϵ -closure (q_0) = $\{q_0, q_1, q_2\}$ (new initial state for NFA, without ϵ -transition)

- (iv) Rest of the states are
 ϵ -closure (q_1) = $\{q_1, q_2\}$ (New state)
 ϵ -closure (q_2) = $\{q_2\}$ (New state)
- (v) The final states of NFA without ϵ -transition are all those new states which contains final state of NFA with ϵ -transition as member.
 So $\{q_0, q_1, q_2\}$, $\{q_1, q_2\}$ and $\{q_2\}$ all are final states.
 So if NFA without ϵ -transition is

$$M' = (Q', \Sigma, \delta', q_0', F')$$

$$Q' = (\{q_0, q_1, q_2\}, \{q_1, q_2\}, \{q_2\})$$

$$q_0' = \{q_0, q_1, q_2\}$$

$$F' = (\{q_0, q_1, q_2\}, \{q_1, q_2\}, \{q_2\})$$

Step 2. Now we have to decide δ' to find out the transitions as follows :
 Very carefully consider each old machine transition in each row, you can ignore ϕ entries and ϵ -transitions column. In the old machine $\delta(q_0, a) = q_0$ thus in new machine i.e. in M' , $\delta'(\{q_0, q_1, q_2\}, a) = \{q_0, q_1, q_2\}$ this is just because the new machine accepts the same language as the old machine and must at least have the same transitions for the new state names. We can allow interpret it as follows :

$$\begin{aligned} \delta'(\{q_0, q_1, q_2\}, a) &= \epsilon\text{-closure}(\delta(q_0, q_1, q_2), a) \\ &= \epsilon\text{-closure}(\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)) \\ &= \epsilon\text{-closure}(\{q_0\} \cup \phi \cup \phi) \\ &= \epsilon\text{-closure}(\{q_0\}) \\ &= \{q_0, q_1, q_2\} \end{aligned}$$

$$\begin{aligned} \text{Similarly } \delta'(\{q_0, q_1, q_2\}, b) &= \epsilon\text{-closure}(\delta(q_0, q_1, q_2), b) \\ &= \epsilon\text{-closure}(\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b)) \\ &= \epsilon\text{-closure}(\phi \cup q_1 \cup \phi) \\ &= \epsilon\text{-closure}(\{q_1\}) \\ &= \{q_1, q_2\}. \\ \delta'(\{q_0, q_1, q_2\}, c) &= \epsilon\text{-closure}(\delta(q_0, q_1, q_2), c) \\ &= \epsilon\text{-closure}(\delta(q_0, c) \cup \delta(q_1, c) \cup \delta(q_2, c)) \\ &= \epsilon\text{-closure}(\phi \cup \phi \cup q_2) \\ &= \epsilon\text{-closure}(\{q_2\}) \\ &= \{q_2\}. \end{aligned}$$

In the same fashion we can write

$$\begin{aligned} \delta'(\{q_1, q_2\}, a) &= \phi \\ \delta'(\{q_1, q_2\}, b) &= \{q_1, q_2\} \\ \delta'(\{q_1, q_2\}, c) &= \{q_2\} \\ \delta'(\{q_2, a\}) &= \phi \\ \delta'(\{q_2, b\}) &= \phi \\ \delta'(\{q_2, c\}) &= \{q_2\} \end{aligned}$$

So transition table for NFA without ϵ -transition will be as follows :

δ'/Σ	a	b	c
$\rightarrow * \{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
$* \{q_1, q_2\}$	ϕ	$\{q_1, q_2\}$	$\{q_2\}$
$* \{q_2\}$	ϕ	ϕ	$\{q_2\}$

Let us say $\{q_0, q_1, q_2\}$ as q_x
 $\{q_1, q_2\}$ as q_y
 $\{q_2\}$ as q_z
 and

So transition diagram is as follows :

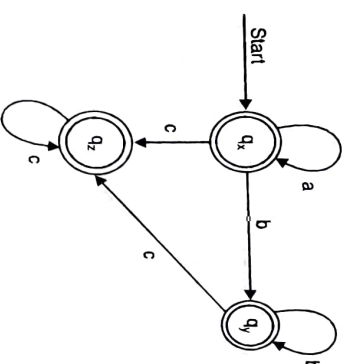


Fig. 2.45. NFA without ϵ -transitions.
 Let us see one more example.

Example 2.19. Consider the NFA given by following diagram

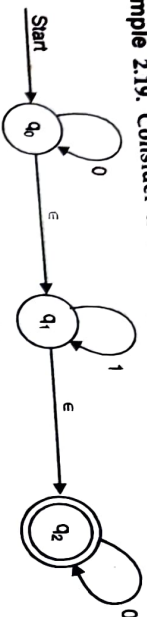


Fig. 2.46.

Find the equivalent NFA without ϵ -transitions.

Solution. Let NFA without ϵ -transition be $M' = (Q', \Sigma, q'_0, \delta', F')$

So as we see in example 2.19.

$$Q' = (\epsilon\text{-closure}(q_0), \epsilon\text{-closure}(q_1), \epsilon\text{-closure}(q_2)) \\ = (\{q_0, q_1, q_2\}, \{q_1, q_2\}, \{q_2\}) \equiv (q_a, q_b, q_c)$$

$$\Sigma = \{0, 1\}$$

$$F' = (q_a, q_b, q_c) \text{ as } q_a \rightarrow \{q_0, q_1, q_2\}$$

$$q_b \rightarrow \{q_1, q_2\}$$

$$q_c \rightarrow \{q_2\}$$

and

$$\delta'(q_a, 0) = q_a$$

$$\delta'(q_a, 1) = q_b$$

$$\delta'(q_b, 0) = q_c$$

$$\delta'(q_b, 1) = q_b$$

$$\delta'(q_c, 0) = q_c$$

$$\delta'(q_c, 1) = \phi$$

So transition table is as follows :

δ'/Σ	0	1
$\rightarrow^* q_a$	q_a	q_b
$^* q_b$	q_c	q_b
$^* q_c$	q_c	ϕ

and transition diagram will be as follows :

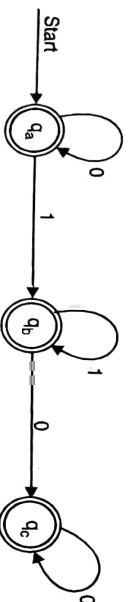


Fig. 2.47. NFA without ϵ -transitions.

2.15. MINIMIZATION OF DFA'S

Any DFA defines a unique language, but the converse is not true. For a given language, there may be many DFA's that accepts it. To understand the minimization first of all one should be very clear about the dead state, inaccessible states and indistinguishable states, so let us discuss them first.

Dead state. All those non final states which transit to itself for all input symbols in Σ , are called Dead state.

Let

$$\Sigma = \{a, b\}$$

then q is said to be dead state.

If q is not the member of F

and

$$\delta(q, a) = q \\ \delta(q, b) = q$$

i.e.,

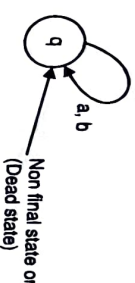


Fig. 2.48.

in the following Fig. 2.49 q_4 is dead state.

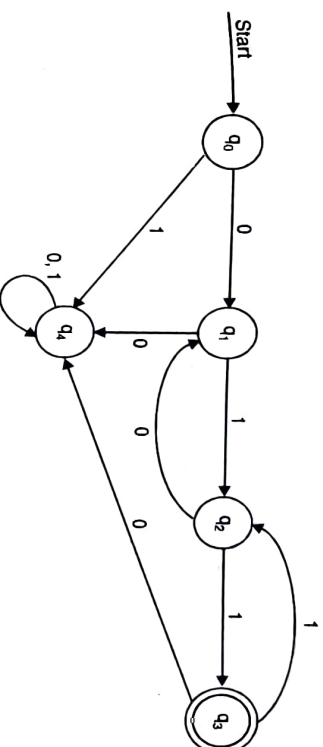


Fig. 2.49.

Inaccessible states. All those states which can never be reached from initial state are called inaccessible states. For example :

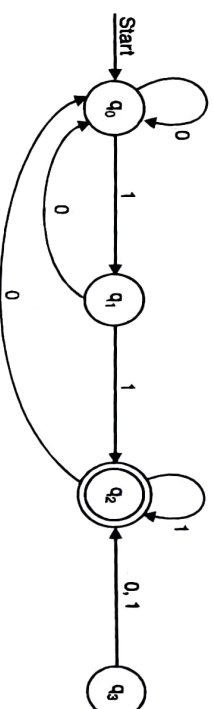


Fig. 2.50.

In Fig. 2.50 q_3 is the state which is not reachable from q_0 (initial state). So q_3 is the unreachable state.

Indistinguishable states. Two states q and r of DFA are called indistinguishable if

$$\hat{\delta}(q, w) \in F \Rightarrow \hat{\delta}(r, w) \in F$$

$$\hat{\delta}(q, w) \notin F \Rightarrow \hat{\delta}(r, w) \notin F$$

$$w \in \Sigma^*$$

and

for all

Distinguishable states. Two states q_1 and q_2 of DFA are called distinguishable if

$$\hat{\delta}(q_1, w) \in F \text{ and } \hat{\delta}(q_2, w) \notin F \text{ or vice versa; for all } w \in \Sigma^*.$$

Now let us discuss the steps involve in the minimization process of a given DFA. Here we will consider an example.

Example 2.20. Minimize the DFA given below.

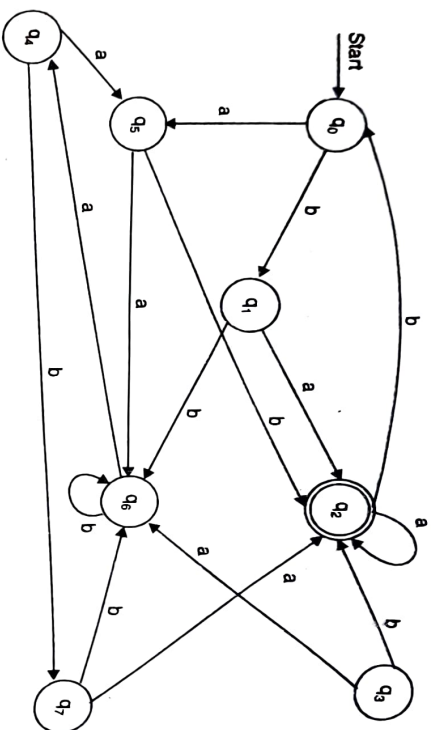


Fig. 2.51.

Solution. Let us discuss the solution step by step.

Step 1. Remove the unreachable states from the given DFA. The DFA given in Fig. 2.51, q_3 is unreachable state so remove it.

Step 2. Draw the transition table for rest states, after removing q_3 (unreachable states).

δ/Σ	a	b
$\rightarrow q_0$	q_5	q_1
q_1	q_2	q_6
$*q_2$	q_2	q_0
q_4	q_5	q_7
q_5	q_6	q_2
q_6	q_4	q_6
q_7	q_2	q_6

δ/Σ	a	b
$\rightarrow q_0$	q_5	q_1
q_1	q_2	q_6
q_4	q_5	q_7
q_5	q_6	q_2
q_6	q_4	q_6
q_7	q_2	q_6

Set-1 (say)

(ii) Other set contains only those rows, which starts from final states. i.e.

q_2 q_2 q_0 Set-2 (say)

Step 3. Now divide rows of transition table in two sets as :

(i) One set contains only those rows, which starts from non-final states i.e.

Step 4. Apply step 4 on both the sets individually. Now first consider set-1

q_0	q_5	q_1	$\rightarrow$	row-1
q_1	q_2	q_6	$\rightarrow$	row-2
q_4	q_5	q_1	$\rightarrow$	row-3
q_5	q_6	q_2	$\rightarrow$	row-4
q_6	q_4	q_6	$\rightarrow$	row-5
q_7	q_2	q_6	$\rightarrow$	row-6

row 2 and row 6, are similar since q_1 and q_7 transit to same states on a and b . So skip one of them let skip row 6, then q_7 is replaced by q_1 in the rest. Write the rest as follows :

q_0	q_5	q_1	$\rightarrow$	row-1
q_1	q_2	q_6	$\rightarrow$	row-2
q_4	q_5	q_1	$\rightarrow$	row-3
q_5	q_6	q_2	$\rightarrow$	row-4
q_6	q_4	q_6	$\rightarrow$	row-5

Step 5. Repeat step 4 on the rest of set 1 till you are finding similar rows and write rest, now this set is minimised.

q_0	q_5	q_1
q_1	q_2	q_6
q_4	q_5	q_1
q_5	q_6	q_2
q_6	q_4	q_6

as row-1 and row-3 are similar

Now minimised set is

q_0	q_5	q_1
q_1	q_2	q_6
q_5	q_6	q_2
q_6	q_0	q_6

See, now there are no two rows are similar.

Step 6. Do same process for set 2 as :

q_2 q_2 q_0

So only single row, already minimised.

Step 7. Now combine set 1 and set 2 as :

δ/Σ	a	b
$\rightarrow q_0$	q_5	q_1
q_1	q_2	q_6
q_5	q_6	q_2
q_6	q_0	q_6
$*q_2$	q_2	q_6

Now it is the transition table of minimised DFA.
Transition diagram of minimised DFA :

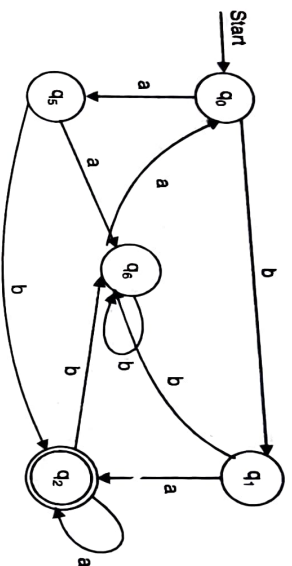


Fig. 2.52.

Example 2.21. Minimise the given DFA.

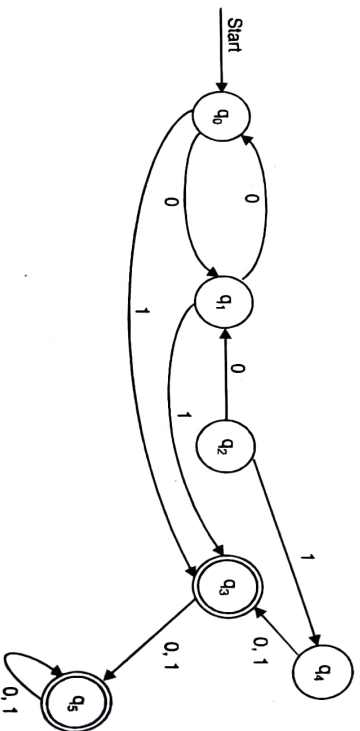


Fig. 2.53.

Solution. DFA given in Fig. 2.53 is having q_2 and q_4 as unreachable states, so remove them and draw the transition table for rest of the states.

δ/Σ	0	1
set 1 $\left[\begin{array}{l} \rightarrow q_0 \\ q_1 \end{array} \right]$	q_1 q_0	q_3 q_3
set 2 $\left[\begin{array}{l} *q_3 \\ *q_5 \end{array} \right]$	q_5 q_3	q_5 q_3

δ/Σ	0	1
$\rightarrow q_0$	q_1	q_3
q_1	q_0	q_3
$*q_3$	q_3	q_3

Set 1 has no, two similar rows so we have to minimise only set 2. Now transition table for minimised DFA is as follows :

47 **++ Finite Automata**
Transition diagram is as follows :

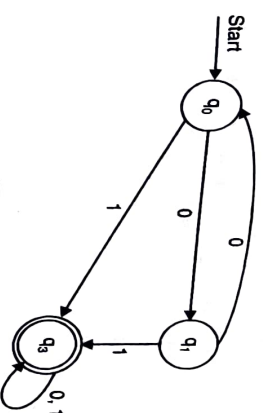


Fig. 2.54.

Example 2.22. Design a DFA for the language

Solution. Let DFA be $M = \{w : n_a = 1, w \in (a, b)^*\}$
 $M = (Q, \Sigma, \delta, q_0, F)$

When we analyse the language, then it contains a set of string, every string contains exactly one 'a' only, that is $a, bbb \dots a, bbb \dots abbb \dots$ and $abb \dots$. Then transition diagram will be as follows :



Fig. 2.55.

Here q_2 is dead state

$Q = \{q_0, q_1, q_2\}$
initial state = $\{q_0\}$
 $\Sigma = \{a, b\}$
 $F = \{q_1\}$

Example 2.23. Design a DFA for the language

$L = \{w : n_a(w) \geq 1, w \in (a, b)^*\}$.

Solution. Given language is

$L = \{w : n_a(w) \geq 1, w \in (a, b)^*\}$

Clearly language is the set of strings, in which every string containing at least one 'a', any where in the string such as $a, bbb \dots a, aaaa, bababa, aabbb \dots$

Let DFA be

$M = (Q, \Sigma, \delta, q_0, F)$
 $\Sigma = \{a, b\}$
 $Q = \{q_0, q_1\}$
 $F = \{q_1\}$

Transition Diagram is

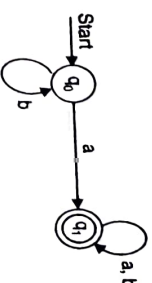


Fig. 2.56.

Example 2.24. Design the deterministic finite automata for the given language

$$L = \{w : n_a(w) \leq 3, w \in \{a, b\}^*\}.$$

Solution. Let us analyse the language first, clearly language L contains set of strings in which maximum number of a 's can be three only. It means strings included no a 's, one a , two a 's and three a 's are the part of the language, empty string is also included.

Strings may be $\epsilon, a, aa, aaa, bbbb, \dots, ba, bab, baaabbb, \dots$

Let DFA be

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$\Sigma = \{a, b\} \text{ given}$$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$F = \{q_0, q_1, q_2, q_3\}$$

Transition diagram is shown in Fig. 2.57.

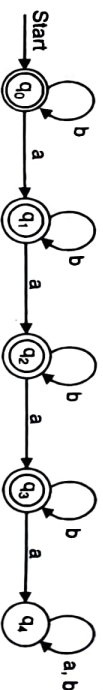


Fig. 2.57.

Here q_4 is dead state.

Example 2.25. Design a DFA for the given language

$$L = \{w : n_a(w) \geq 1, n_b(w) = 2, w \in \{a, b\}^*\}.$$

Solution. Let us analyse the language first, since here conditions are defined as $n_a(w) \geq 1$ and $n_b(w) = 2$ means every string of the language must contain at least one ' a ' exactly two ' b 's such as $abb, bab, baaab, \dots, b, aabab, \dots$ $baab, bba$ and many more which are satisfying both the conditions.

Let us assume that DFA be $m = (Q, \Sigma, \delta, q_0, F)$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8\}$$

$$F = \{q_3, q_8\}$$

$$\Sigma = \{a, b\}$$

where

Transition diagram is shown in Fig. 2.58.

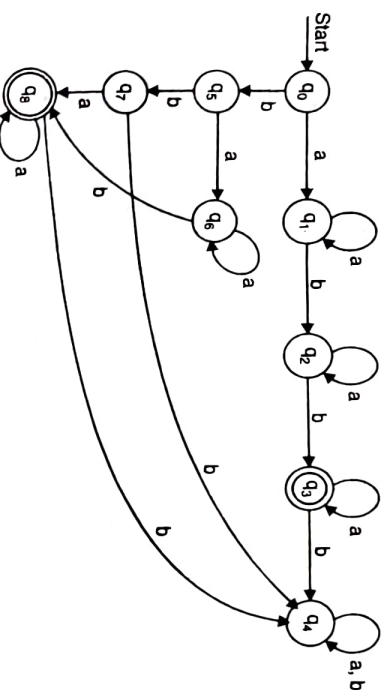


Fig. 2.58.

Example 2.26. Design a DFA for the language

$$L = \{w : n_a(w) = 2, n_b(w) > 2, w \in \{a, b\}^*\}.$$

Solution. Given language is $L = \{w : n_a(w) = 2, n_b(w) > 2, w \in \{a, b\}^*\}$. Now it is clear from $n_a(w) = 2$ and $n_b(w) > 2$ that strings, which are belongs to language must satisfied both the condition simultaneously that is every string must contain exactly two a 's in it (in any position) and number of b 's must be more than two (in any position), such as $abbbb, abbb, abbb, bbbba$.

Let DFA be

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11}, q_{12}\}$$

$$\Sigma = \{a, b\}$$

$$F = \{q_5\}$$

Transition diagram is shown in Fig. 2.59.

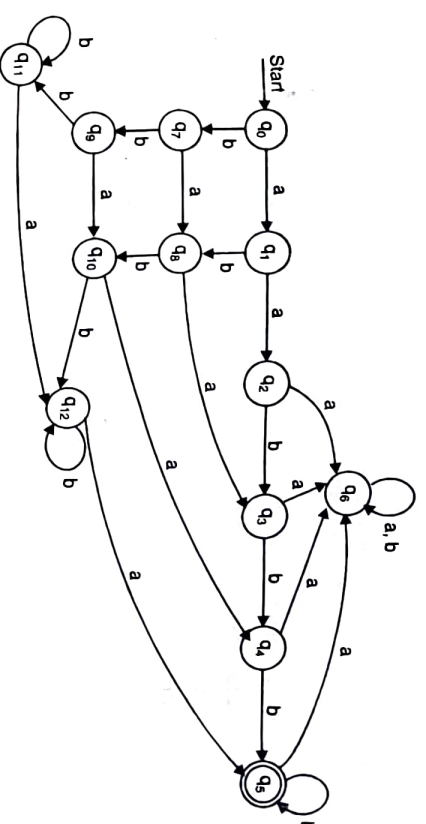


Fig. 2.59.

Example 2.27. Given DFA is

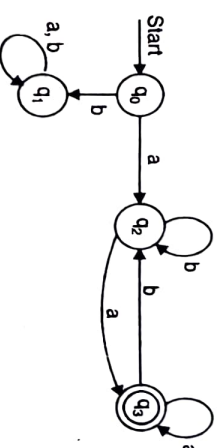


Fig. 2.60.

(i) Give the language of DFA shown in Fig. 2.60.

(ii) Prove that there exist a DFA for $\bar{L}$ if L is the language of given DFA.

Solution. (i) Let

$$M = (Q, \Sigma, \delta, q_0, F) \text{ be given DFA}$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$F = \{q_3\}$$

$$\Sigma = \{a, b\}$$

Here q_1 is the dead state, clearly no string of the language will start from b character. So every string starts with 'a' and end with 'a'.

(ii) Now make q_3 as non-final state and q_0, q_1, q_2 as final states and prove the new automaton obtained accepts $\bar{L}$ i.e. it does not accepts awa . Now it can be show through the following transition diagram.

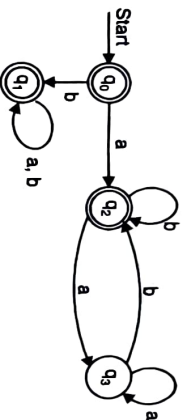


Fig. 2.61.

This automaton accepts $b(a, b)^* \in$, and $ab^*aa^*bb^*$ or ab^* . Clearly it accepts all strings which starts with either b or if it starts with an a , it ends with ab^* and if it encounters any a after b^* , it must end with b .

So, this automaton accepts $\bar{L}$.

Example 2.28. Show that if $M = (Q, \Sigma, \delta, q_0, F)$ and $\bar{M} = (Q, \Sigma, \delta, q_0, Q-F)$ are two DFA's, then $L(\bar{M}) = L(M)$.

Solution. Let there be an automaton

$$M = (Q, \Sigma, \delta, q_0, F)$$

where Q = non-empty set of state in automaton

Σ = non-empty set of input symbols

δ = transition rule (function) from $Q \times \Sigma$ to Q

q_0 = start state (where $q_0 \in Q$)

F = non-empty set of final states.

If we consider another automaton

$$\bar{M} = (Q, \Sigma, \delta, q_0, Q-F)$$

where all Q, Σ, δ and q_0 are same as previous, but the final states are the other state than F , then the new automaton can not reach finally at the state (F) in this automaton as it used to be in the automaton M . So, this automaton will accept other language, but it will not accept the language which is accepted by M .

So $L(\bar{M}) = \bar{L(M)}$

We can also understand this by an example.

Let us consider a DFA for the language

$$L = \{a^n b^0 : n \geq 0\}.$$

Transition diagram as follows :

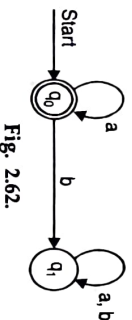


Fig. 2.62.

Let say

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1\}$$

$$q_0 = q_0$$

$$F = \{q_0, q_1\} - \{q_0\} = \{q_1\}$$

Now making
So we will get



Fig. 2.63.

This automaton (shown in Fig. 2.63) accepts all strings, except $a^n b^0, n \geq 0$.

Example 2.29. Design a DFA for the language

$$L = \{ab^5 w b^4 : w \in \{a, b\}^*\}.$$

Solution. Let DFA be

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_{10}, q_{11}\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_{10}\}$$

where

Transition diagram is following :

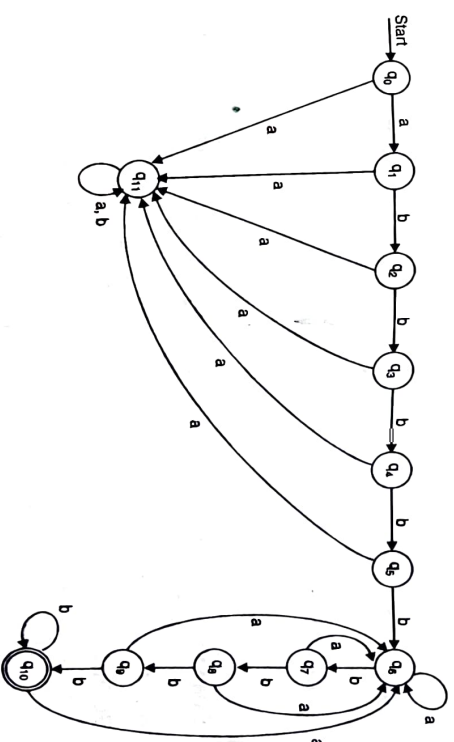


Fig. 2.64.

Example 2.30. Design a DFA for the language

$$L = \{w_1 ab w_2 : w_1, w_2 \in \{a, b\}^*\}.$$

Solution. It is clearly known after analysis of language L that every string at least one ab in string at any position.

Let DFA be

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2\}$$

where



$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_2\}$$

Transition diagram for the given language is shown in Fig. 2.65, as follows :

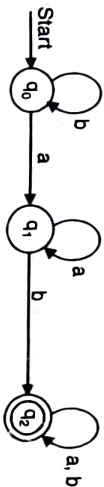


Fig. 2.65.

Example 2.31. Give the description of the language for the DFA, shown in Fig. 2.66.

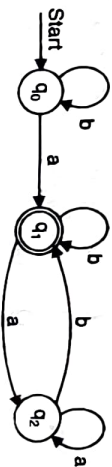


Fig. 2.66.

Solution. Given DFA $M = (Q, \Sigma, \delta, q_0, F)$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_1\}$$

Language of DFA can be defined as follows :

$$L(M) = \{b^*a\{wb\}^* : w \in \{a, b\}^*\}$$

$$L(M) = \{b^*a\{a^*b\}^*\}.$$

or

Example 2.32. Design a deterministic finite automation over alphabet $\Sigma = \{a, b\}$, such that every string, accepted by automation contains no runs of length less than four.

Solution. Let us first understand the problem properly, every string contains no runs of length less than four means that string may start either from a or from b but, automation will accept only those strings which contains a 's or b 's in the runs of four such as $aaaa$ or $bbbb$.

Let DFA be

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_4, q_8\}$$

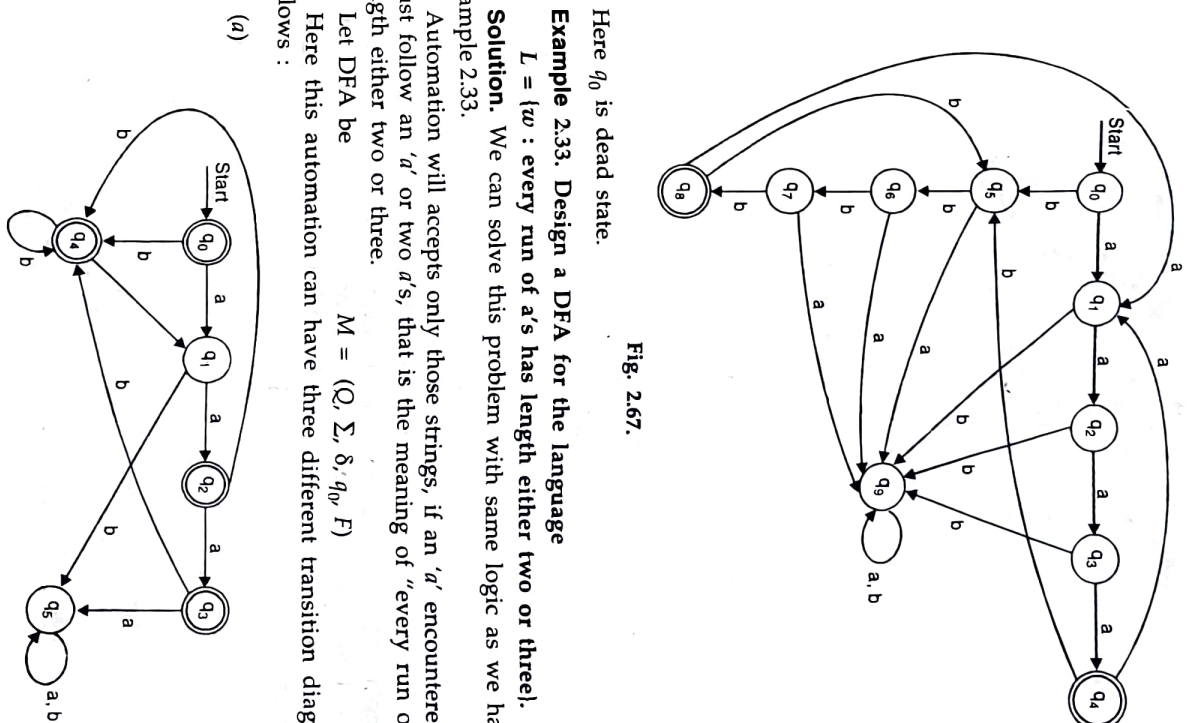


Fig. 2.67.

Here q_0 is dead state.

Example 2.33. Design a DFA for the language

$$L = \{w : \text{every run of } a\text{'s has length either two or three}\}.$$

Solution. We can solve this problem with same logic as we have solve Example 2.33.

Automation will accepts only those strings, if an ' a ' encountered then it must follow an ' a ' or two a 's, that is the meaning of "every run of a 's has length either two or three.

Let DFA be

$$M = (Q, \Sigma, \delta, q_0, F)$$

Here this automation can have three different transition diagrams, as follows :

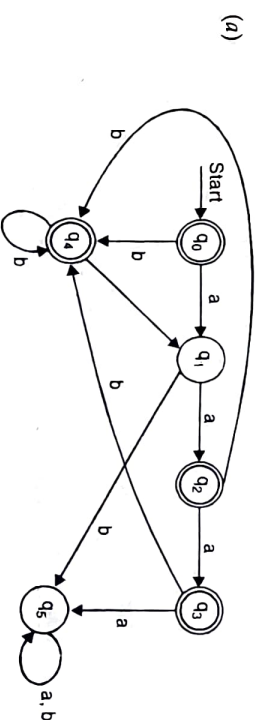


Fig. 2.68.

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_2, q_3, q_4, q_0\}$$

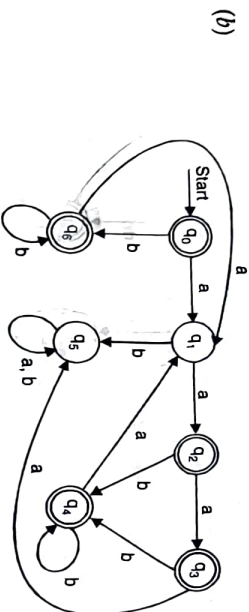


Fig. 2.69.

$$\begin{aligned} Q &= \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\} \\ \Sigma &= \{a, b\} \\ q_0 &= \{q_0\} \\ F &= \{q_2, q_3, q_4, q_6, q_0\} \end{aligned}$$

Here

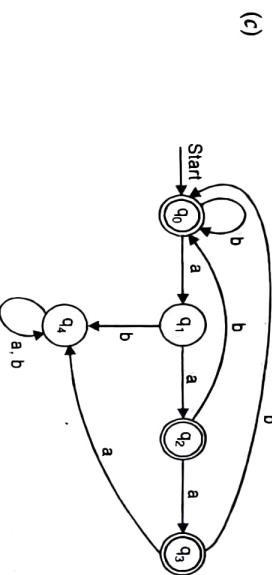


Fig. 2.70.

$$\begin{aligned} Q &= \{q_0, q_1, q_2, q_3, q_4\} \\ \Sigma &= \{a, b\} \\ q_0 &= \{q_0\} \\ F &= \{q_2, q_3, q_0\} \end{aligned}$$

Here

Example 2.34. Design a DFA, which accepts strings, in which every 00 is followed immediately by a 1. For example, the strings 001, 0010, 00100111001 are in the language, but 0001 and 00100 are not.

Solution. Let DFA be $M = (Q, \Sigma, \delta, q_0, F)$
 $Q = \{q_0, q_1, q_2, q_3, q_4\}$
 $\Sigma = \{0, 1\}$
 $q_0 = \{q_0\}$
 $F = \{q_0, q_1\}$

Transition diagram is show in Fig. 2.71.

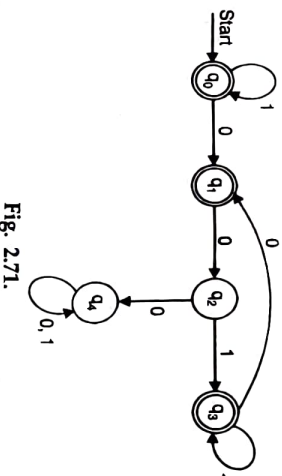


Fig. 2.71.

Here q_4 is dead state.

Example 2.35. Design a DFA for the language, contains strings in which left most symbol differ from right most symbol. Σ is given $\{0, 1\}$.

Solution. Here problem is straight forward. Language is the set of strings such as 01110001101, 01, 10, or in general we can say that strings are either 0 w 1 or 1 w 0 where $w \in \{0, 1\}^*$.

Let DFA be

$$\begin{aligned} M &= (Q, \Sigma, \delta, q_0, F) \\ Q &= \{q_0, q_1, q_2, q_3, q_4\} \\ \Sigma &= \{0, 1\} \\ q_0 &= \{q_0\} \\ F &= \{q_2, q_4\} \end{aligned}$$

Transition diagram is described in Fig. 2.72.

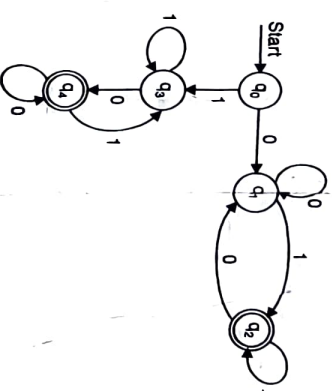


Fig. 2.72.

Example 2.36. Design a deterministic finite automaton which accepts set of strings such that every string containing 00 as a substring but not 000 as sub-string.

Solution. Let DFA be $M = (Q, \Sigma, \delta, q_0, F)$
 $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$
 $\Sigma = \{0, 1\}$
 $q_0 = \{q_0\}$
 $F = \{q_2, q_3, q_5\}$

Transition diagram is defined in Fig. 2.73 as follows :

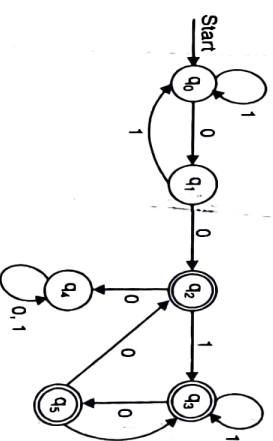


Fig. 2.73.

Here q_4 is dead state.

Example 2.37. Construct a DFA that accepts strings on $\{0, 1\}$, if and only if the value of the string, interpreted as a binary representation of an integer, is zero module five. For example, 0101 and 1111, representing the integers 5 and 15, respectively, are to be accepted.

Solution. Let us understand problem first. We have to design a automaton which accepts those binary strings, when we interpreted them in integer then those integers are divisible by 5 that means after dividing 5 remainder is 0, it is the only meaning of the "is zero module five". So clearly integer interpretation should be 0, 5, 15, 20, 25, ... So binary strings which are accepted by automaton are 0, 101, 1111 ...

Let DFA be

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 = |q_0|$$

$$F = \{q_0\}$$

and finally transition diagram is defined as follows :

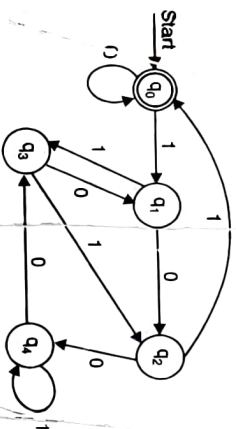


Fig. 2.74.

Example 2.38. Show that if L is regular, so is $L - \{\epsilon\}$.

Solution. Let us assume that L is regular and w is the string such that $w \in L$. So automaton will be



Fig. 2.75.

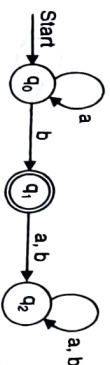
If we substracted ϵ from L then we have $L - \{\epsilon\}$.
Transition diagram can be defined as :



Fig. 2.76.

So, $L - \{\epsilon\}$ is also regular.

Example 2.39. Here DFA given in Fig. 2.77 for the language L find the DFA for L^2 .

Fig. 2.77. DFA for language L .

Solution. We have given DFA for language L in Fig. 2.77(a).

The language for DFA given is

$$L = \{a^n b : n \geq 0\}$$

Let us assume that DFA for L^2 is

$$M = (Q, \Sigma, \delta, q_0, F)$$

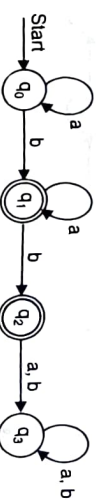
$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = |q_0|$$

$$F = \{q_2\}$$

Then transition diagram for L^2 is defined as follows :

Fig. 2.77(a). DFA for L^2 .

Here q_3 is dead state.

EXERCISES

1. Build an FA that accepts only the words baa , ab and abb and no other strings longer or shorter.
2. For the $\Sigma = \{a, b\}$
 - (i) Build an FA that accepts only those words that have more than four letters.
 - (ii) Build an FA that accepts only those words that have fewer than four letters.
 - (iii) Build an FA that accepts only those words with exactly four letters.
 - (iv) Build an FA that accepts only those words that do not end with ba .
 - (v) Build an FA that accepts only those words that begin or end with a double letter.
3. Describe the language accepted by the following FA's :

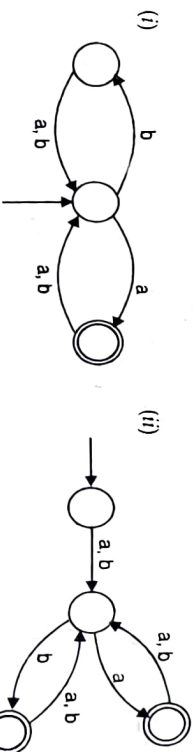


Fig. 2.78.

Fig. 2.79.

4. Convert each of the following NFA's in to FA's



Fig. 2.80.

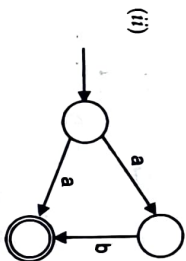


Fig. 2.81.

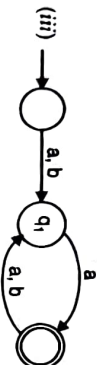


Fig. 2.82.

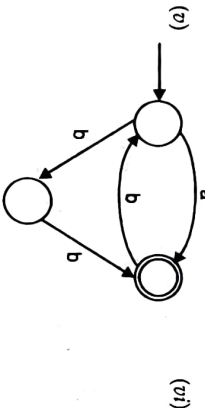


Fig. 2.84.

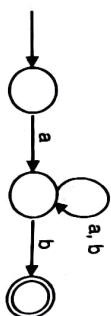


Fig. 2.85.

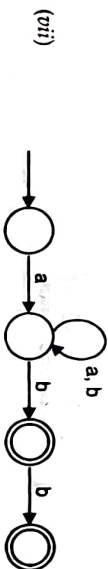
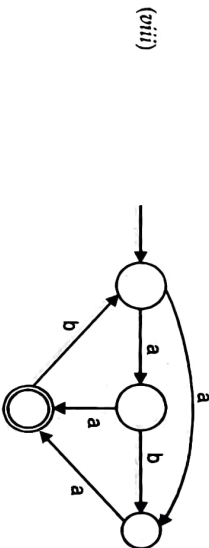


Fig. 2.86.



5. Convert the following NFA- ϵ 's into NFA without ϵ -transitions.

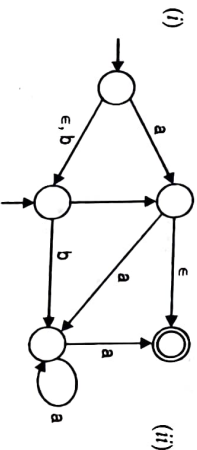


Fig. 2.88.

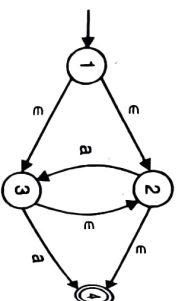


Fig. 2.89.

6. For the language accepted by the following machine, find a different FA with four states. Find an NFA that accepts the same languages has only seven edges (where edges with two labels are counted twice).

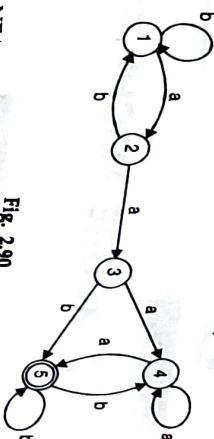


Fig. 2.90.

7. Design a NFA with no more than five states for the set $\{abab^n : n \geq 0\}$.
8. Find an NFA that accepts $\{a\}^*$ and is such that if in its transition graph a single edge is removed, the resulting automaton accepts $\{a\}$.
9. Find an NFA without ϵ -transition and with final state that accepts the set $\{a\}^n : n \geq 1\}$.
10. Write a computer program that produces a minimal DFA for any given DFA.
11. Find the minimal DFA's for the following languages below :
 - (i) $L = \{a^n b^m : n \geq 2, m \geq 1\}$
 - (ii) $L = \{a^n : n \geq 0, n \neq 3\}$
 - (iii) $L = \{a^n : n \neq 2 \text{ and } n \neq 4\}$
